Activity Title:

Building and Integring a Laravel Application with APIs

Objectives

By the end of this activity, students should be able to:

- Set up authentication using Laravel Breeze
- Implement login and registration functionality
- Create a simple REST API
- Consume and display data from a public API
- Understand basic system integration concepts

Activity Overview

Students will build a **simple web application** (e.g., *Student Info Dashboard* or *Mini Weather App*) that:

1. Uses authentication (Login/Register)
2. Provides its own API endpoint
3. Integrates data from a public API

Duration

2–3 sessions (2–3 hours each)

Part 1: Laravel Breeze Setup (Authentication)

Task:

1. Install Laravel project
2. Install Laravel Breeze
3. Setup authentication

Guide:

```
composer create-project laravel/laravel integration-app
cd integration-app

composer require laravel/breeze --dev
php artisan breeze:install

npm install && npm run dev
php artisan migrate
```

Expected Output:

- Working **Login & Registration pages**
- Redirect to dashboard after login

Part 2: Login & Registration Enhancement

Task:

- Add the following fields during registration:
    - Full Name
    - Role (Admin/User)

Challenge:

- Redirect users based on role:
    - Admin → Admin Dashboard
    - User → User Dashboard

Part 3: Creating a Simple API

Task:

Create an API that returns student or user data.

Steps:

1. Create API route in routes/api.php

```
Route::get('/users', [UserController::class, 'index']);
```

2. Create Controller:

```
php artisan make:controller Api/UserController
```

3. Add logic:

```
public function index()
{
   return response()->json(User::all());
}
```

Test:

- Use browser or Postman:

```
http://localhost:8000/api/users
```

Part 4: Consuming a Public API

Task:

Fetch data from a public API (choose one):

- Weather API
- News API
- JSONPlaceholder

Example using JSONPlaceholder:

```
use Illuminate\Support\Facades\Http;

$response = Http::get('https://jsonplaceholder.typicode.com/posts');
```

$data = $response->json();

Output:

- Display API data in a Blade view (table or cards)

Part 5: Integration Task (Main Activity)

Scenario:

Create a **Dashboard that combines:**

- Logged-in user info
- Data from your own API
- Data from a public API

Requirements:

- Show user profile
- Show list of users (from your API)
- Show external data (e.g., posts/weather)

Bonus Challenges

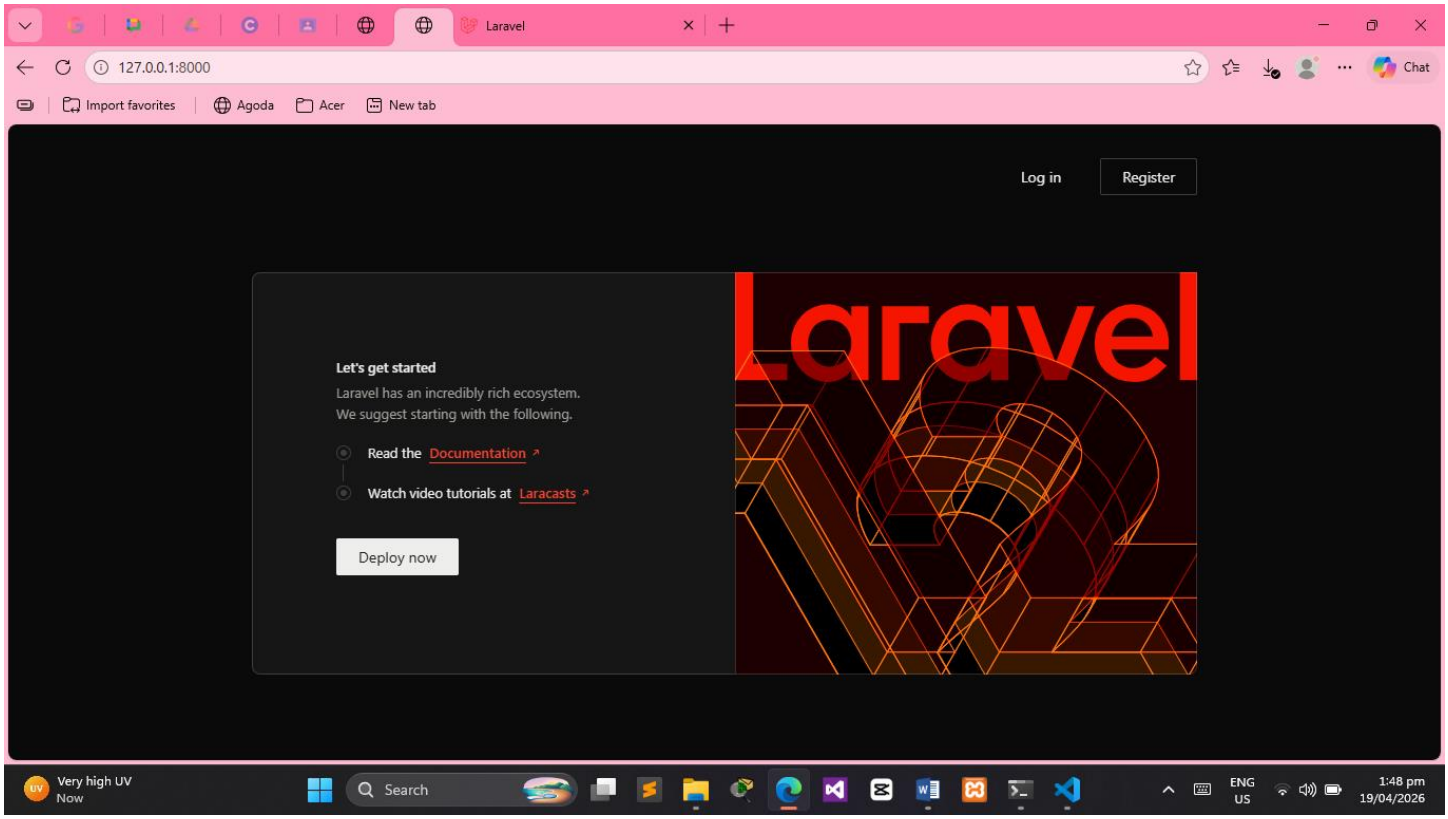
- Add API authentication using tokens (Laravel Sanctum)
- Add search/filter functionality
- Cache API results
- Handle API errors gracefully

Evaluation Criteria

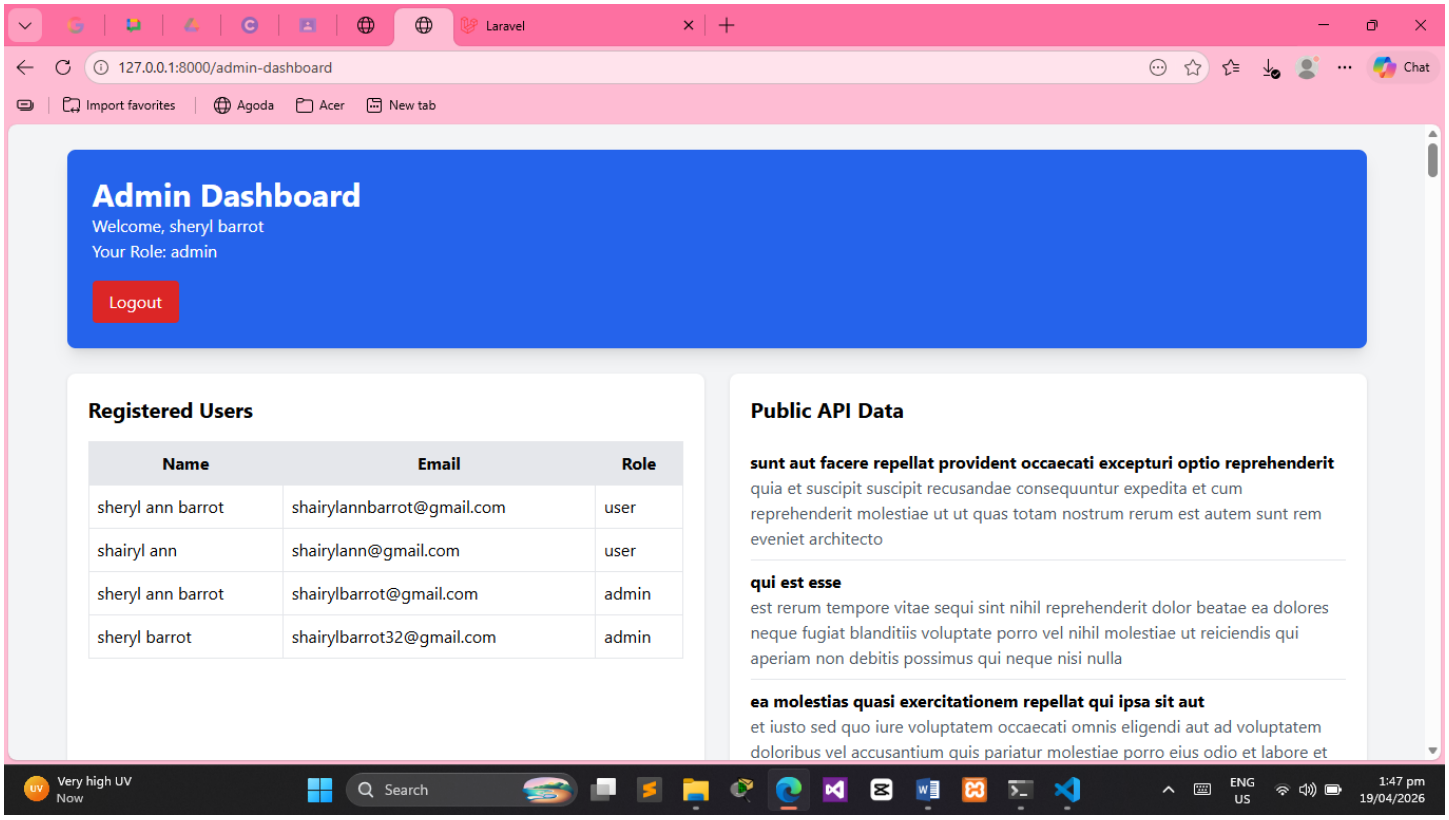
Criteria	**Points**
Laravel Breeze Implementation	20
Login & Registration Functionality	15
API Creation	20
Public API Integration	20
UI & Integration	15
Code Quality	10
Total	**100**

Submission Requirements

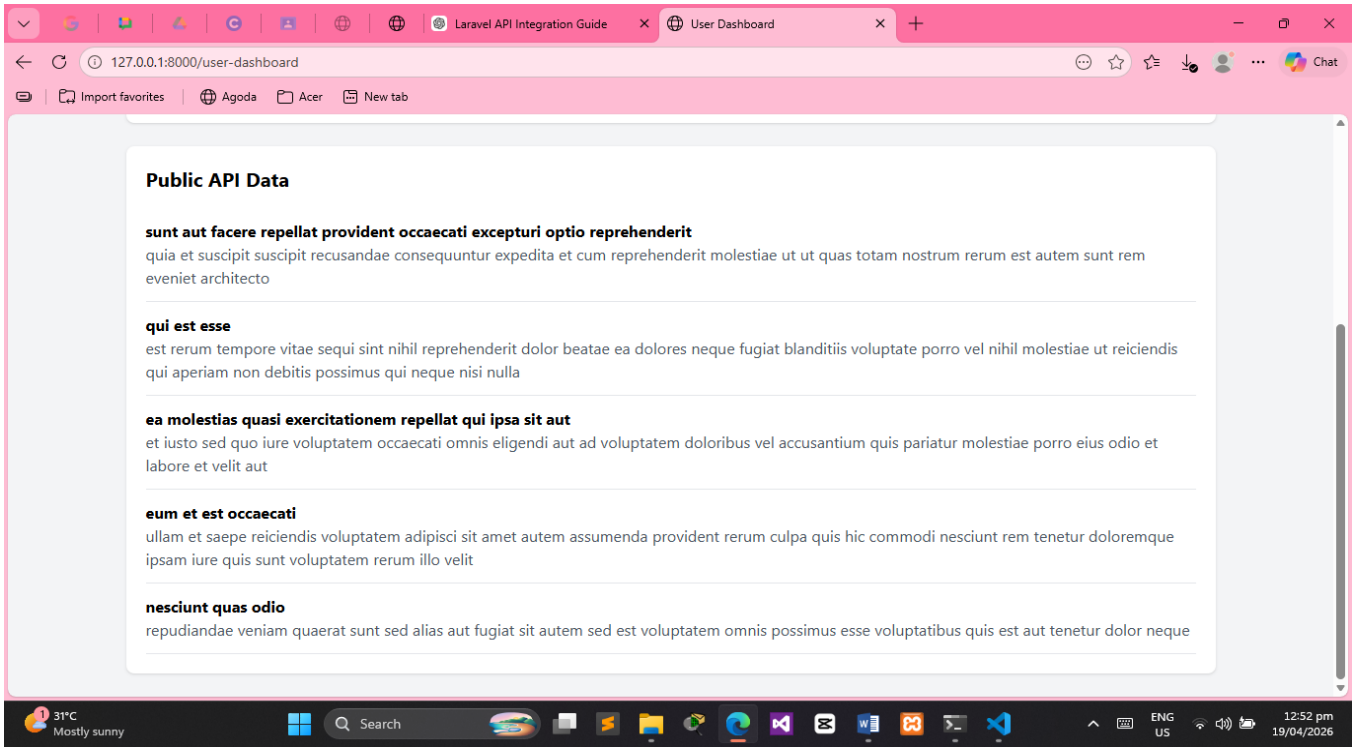
- Screenshots:
    - Login/Register



    - Dashboard



- API output



- **Short documentation (1–2 pages):**
- **System description**

The name of this system is Laravel Authentication and API Integration System. It was built with the Laravel framework and Breeze for authentication.Users can sign up and log in to the system. The system will check the user's role after they log in. It will go to the Admin Dashboard if the user is an admin. If the person is a regular user, it will go to the User Dashboard.

The dashboard displays important data such as the logged-in user's information, a list of users, and information from an external API. This system helps you learn how to use authentication and APIs in a web app.

- **APIs used**

This system uses two APIs:

1. Internal API (Created in Laravel)

Endpoint: /api/users

This API returns all users in JSON format

It is created using Laravel controller and routes

2. Public API

API used: JSONPlaceholder

Link: https://jsonplaceholder.typicode.com/posts

This API provides sample posts with title and body

It is used to display external data inside the dashboard

- **Challenges encountered**

I ran into some problems while making this system. Setting up Laravel Breeze authentication correctly is one of the biggest problems. The login redirect doesn't always work right, and it takes you to the default dashboard instead of the admin or user dashboard.

Another problem is dealing with API data. The data isn't showing up on the dashboard at first because the controller wasn't sending it to the view in the right way.

PowerShell also gave me trouble when I tried to run npm commands because it blocks scripts. I changed the execution policy to fix it. Overall, the system was hard at first, but I get it better now that I've finished all the parts.